

Análise de Desempenho: Adição Vetorial (CPU vs. GPU)

O experimento avaliou o tempo de execução de um algoritmo paralelizado em OpenMP para a adição de vetores ($c[i] = a[i] + b[i]$), processando $N = 500.000.000$ elementos de dupla precisão (double). Este volume de dados equivale à manipulação de aproximadamente 12 GB de memória. Contrariando o senso comum de que aceleradores sempre superam arquiteturas tradicionais, a execução na CPU (distribuída em 8 threads) registrou um tempo de cálculo de meros 0.221s. Por outro lado, a submissão do laço (via **offloading**) para a GPU NVIDIA V100 demandou 1.844s, culminando em um tempo total de execução mais de duas vezes superior ao da CPU.

Métrica de Tempo	Host (CPU)	Device (GPU V100)
Inicialização	0.517s	0.486s
Cálculo / Offload	0.221s	1.844s
Verificação	0.088s	0.067s
Total	0.825s	2.397s

A expressiva queda de desempenho na placa gráfica ilustra de forma didática as limitações associadas a algoritmos de baixa **Intensidade Aritmética**. A operação de soma de dois vetores realiza apenas uma única instrução matemática em oposição a três acessos extensos à memória (leitura das variáveis de origem e escrita do resultado). Ao utilizar a diretiva `#pragma omp target map`, o compilador é instruído a empacotar dados na memória RAM e trafegar os 12 GB através da conexão PCI-Express da placa-mãe até a memória VRAM da GPU, operando subsequentemente a viagem de retorno. Como consequência, o tempo aferido de “cálculo” na V100 retrata, em sua esmagadora maioria, a latência de transferência de dados no barramento. Por se tratar de um algoritmo estritamente **Memory Bound** (limitado por largura de banda), o formidável paralelismo da GPU torna-se ineficaz frente ao gargalo de comunicação física do hardware.